

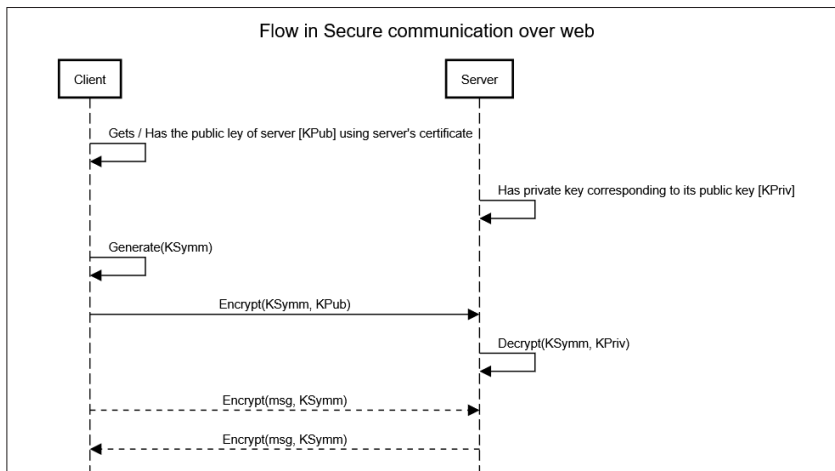


Figure 1: Basics of communication between a web application and the browser

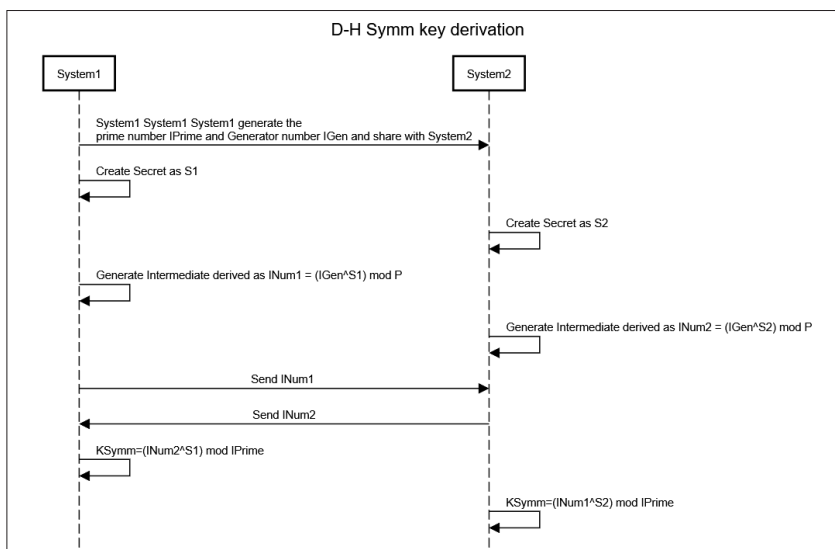


Figure 2: Diffie Hellman algorithm basics

the server and client. It is a case of impersonation and has a huge impact on the server's credibility.

From the time that the private key is compromised to the time the compromise is detected, all the communication can be interpreted by an attacker. This is a very serious security breach that has the severe impact of users losing trust on the services of the application.

An obvious mitigation is to revoke the private key. But this is a daunting task and is not used, unless one is forced to do so. The recommended revocation period is about two years.

Limitations of revoking the private key: The revocation of a key is not a detection technique but a mitigation mechanism. One cannot detect the private key compromise using revocation. Detecting the private key compromise is difficult. So, the duration for which a key compromise incident goes undetected allows an attacker to read and interpret the complete communication between end points.

One of the obvious solutions is to revoke the private key by communicating with the Certificate Authority (CA). This is a daunting task, as the corresponding public key needs

to be redistributed. This seems to be the main reason why it is recommended to refresh the key every two years.

An alternate solution

The alternate solution discussed in this section makes use of the Diffie Hellman algorithm to derive the symmetric key, which will be used for encrypting the traffic for that session. The important step in this algorithm is that rather than exchanging the symmetric key between the endpoints, it is derived at their respective ends.

The sequence diagram given in Figure 2 shows the working of the Diffie Hellman algorithm.

This alternate solution uses the mechanism to generate temporary asymmetric key pairs at both the server and client ends. These temporary keys are used to derive the temporary symmetric key to be used for the encryption of session communication.

Figure 3 gives a diagrammatic representation of the sequence of exchanges expected between end points.

Let us look deeper into the sequence of mechanisms using temporary keys to mitigate the compromise of private keys. Refer to Figure 3 to understand the process step by step.

- Client requests communication with the server.
- Server sends the certificate, which has its public key in it. Client gets the public key [KPub1] of the server.
- Client generates the symmetric key, encrypts it using KPub1 and sends it to the server.
- Server decrypts the message using its private key [KPriv1]. The message can only be decrypted by the server as it has the private key.
- Both the server and client have the symmetric key [KSymm1]. The session uses this KSymm1 to encrypt the communication.
- Client generates the public and private key pair named KPub2 and KPriv2, respectively. It encrypts the KPub2 using KSymm1 as:
 - $\text{Encrypt}(\text{KPub2}, \text{KSymm1})$